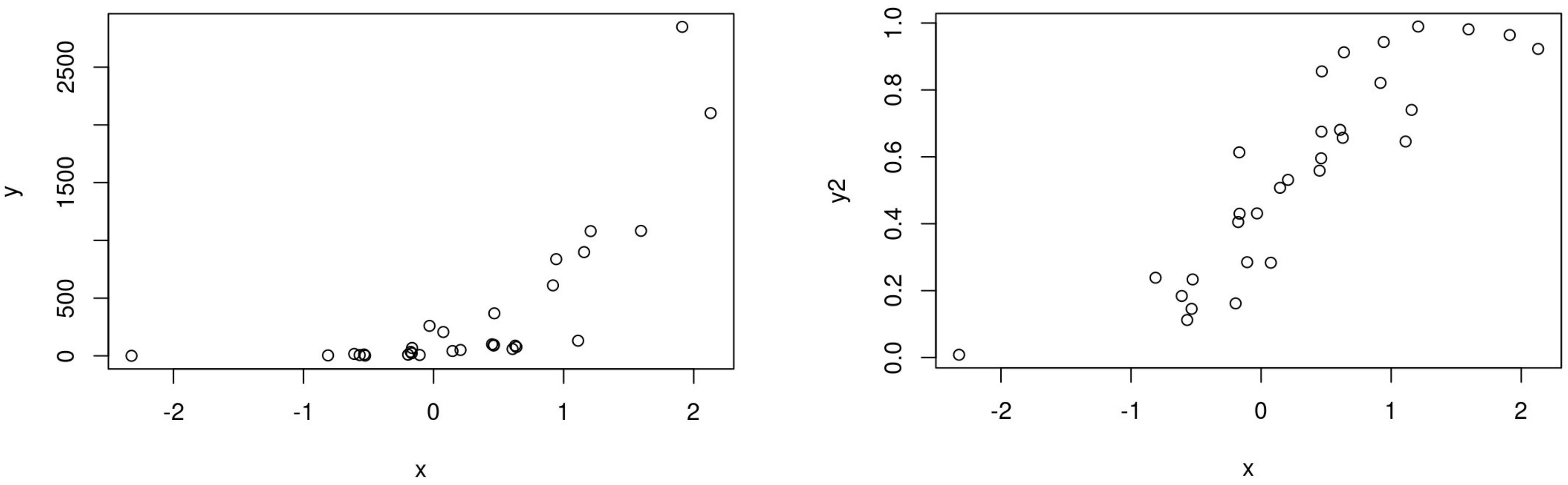
Generalized linear models

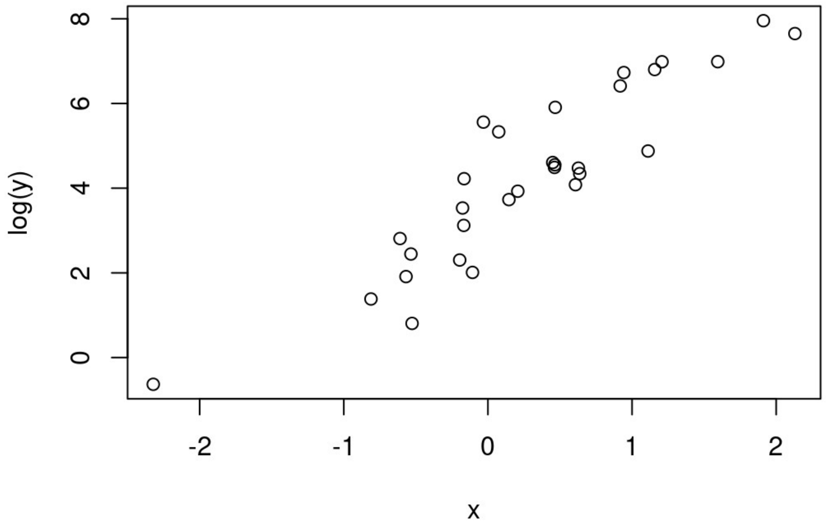
Jan Conradt
06.05.2026

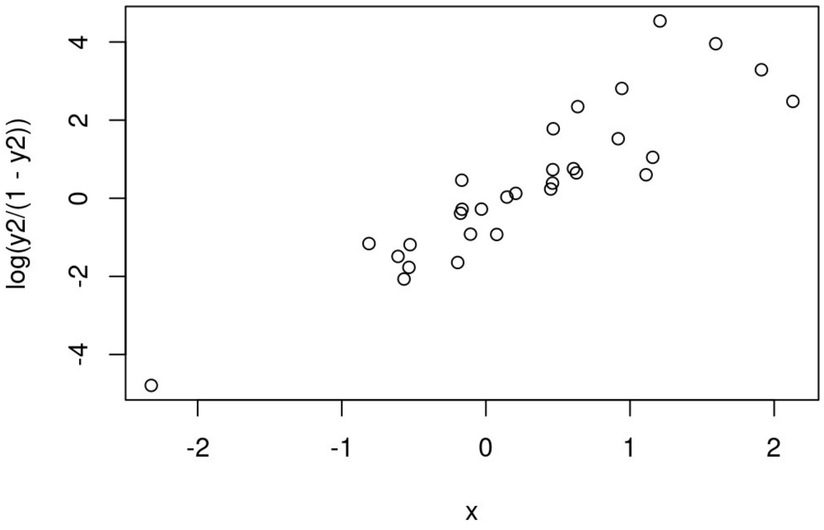
Non-linear relationships...



Non-linear relationships...

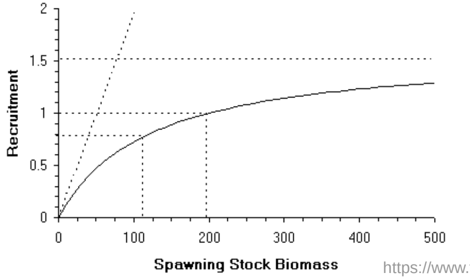
- … can be made linear



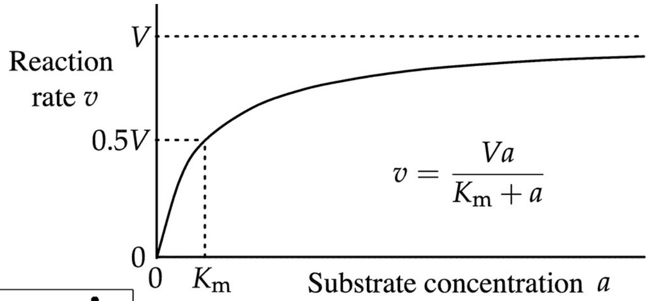


Non-linear relationships...

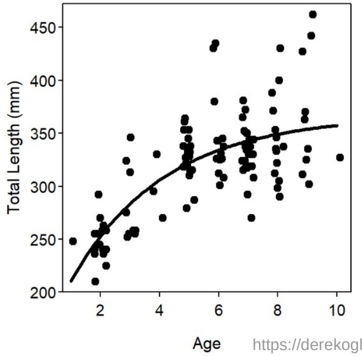
- … can be made linear
-
- Some mechanistic examples:
- → stock-recruitment functions
- → Michaelis-Menten kinetics
- → growth curves



https://www.fao.org/4/w7219e/w7219e05.htm



By Athel cb - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=130422583



https://derekogle.com/NCNRS349/modules/PREP/NOTES/Growth

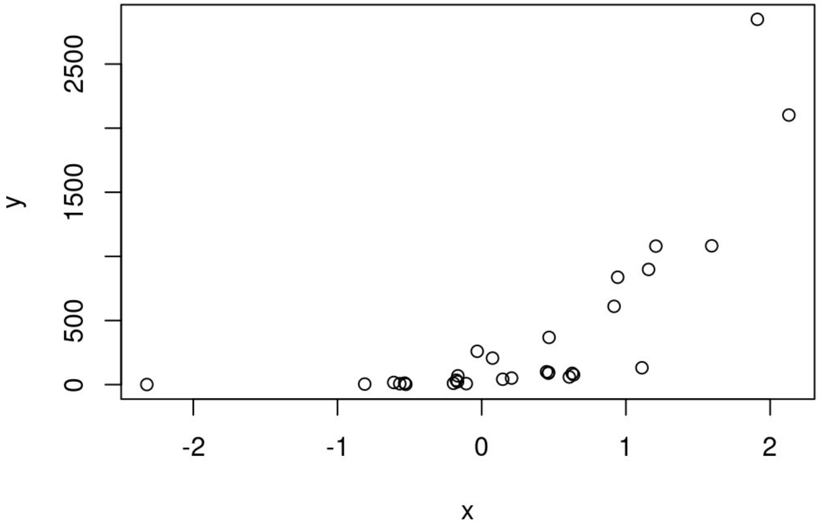
Non-linear relationships...

- … can be made linear
-
- Some mechanistic examples:
- → stock-recruitment functions
- → Michaelis-Menten kinetics
- → growth curves
- ==> rearrange mechanistic equations so they have a more linear look
- ==> often transform y variable (sometimes also x)

Non-linear relationships...

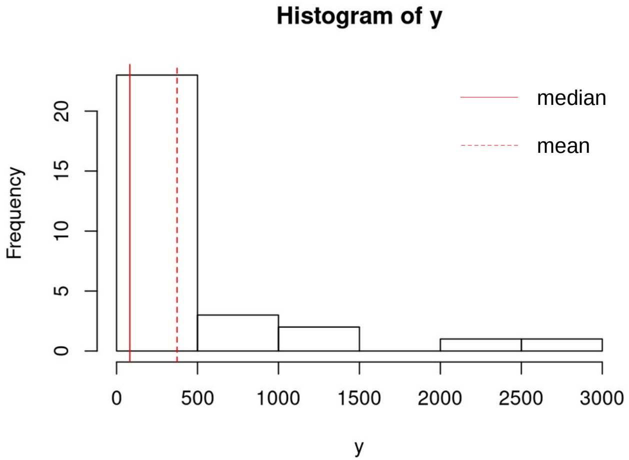
- … can be made linear
-
- But it also works with data-driven models
- → transform y variable
- → or, differently speaking, select a different distribution *family* for y

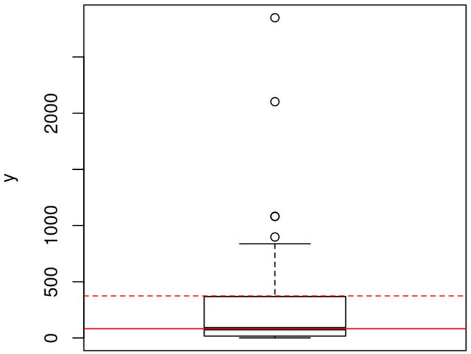
Case #1: log-normally-distributed data



Case #1: log-normally-distributed data

-
- Data are strongly right-skewed
- Median usually << mean
-
- Data follow a log-normal distribution
- Sometimes called a *Poisson* distribution



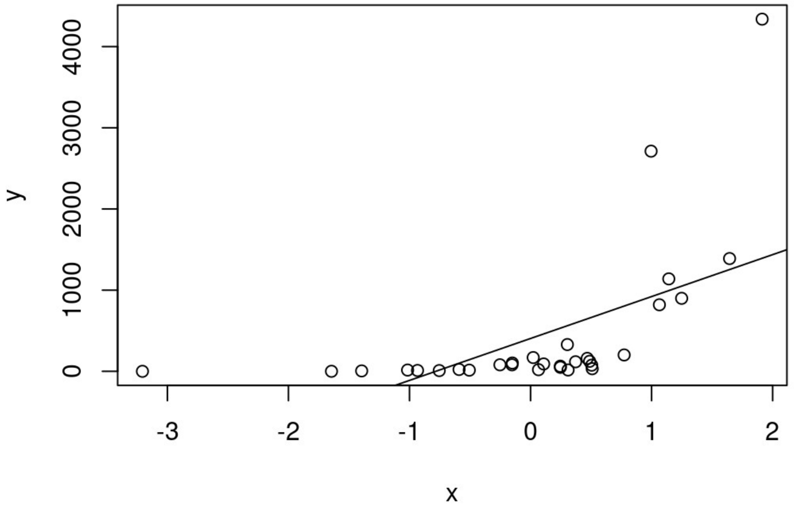


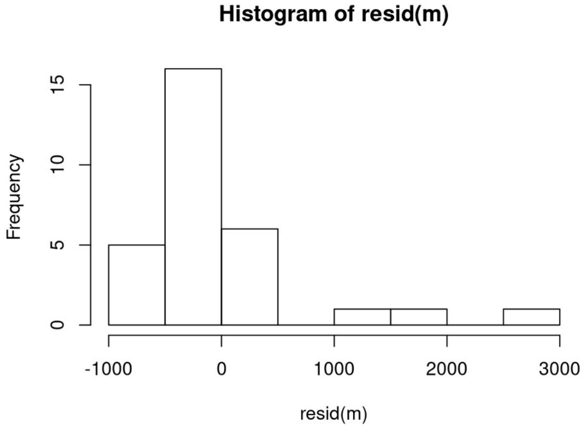
Case #1: log-normally-distributed data

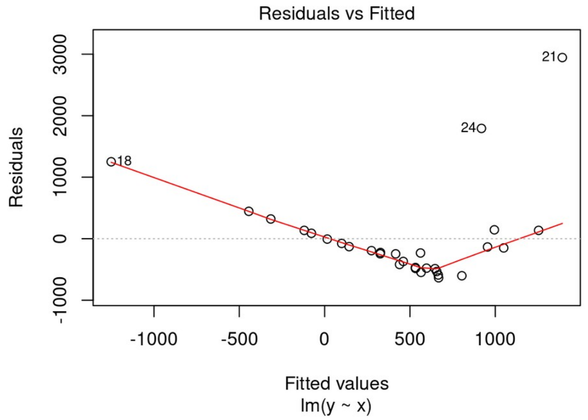
- Data are strongly right-skewed
- Median usually << mean
-
- Typical example: count data
- → many samples with few individuals, few samples with very many individuals

Case #1: log-normally-distributed data

- How to deal with that?







- A normal linear model will give a poor fit

Case #1: log-normally-distributed data

- How to deal with that?
- → logarithmize the y values
- → y’ = log(y) log() is here log_e or ln!
-
-
-

Case #1: log-normally-distributed data

- How to deal with that?
- → logarithmize the y values
- → y' = log(y)   log() is here $\log_e$ or ln!
-
- Removes the lower bound (e.g. log(0.5) = -0.69)
- y cannot be negative or zero (works best with count data)

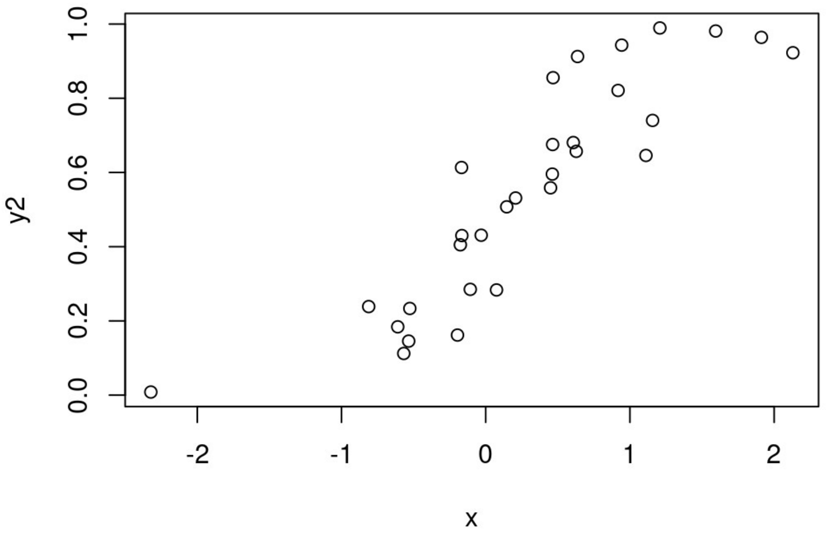
Case #1: log-normally-distributed data

- lm(log(y) ~ x, data = data)
-
- Or you use the “glm()” function instead of “lm”
- → “lm”: “linearized model”
- → “glm”: “generalized linear model”
-
- glm(y ~ x, data = data)
- → ...

Case #1: log-normally-distributed data

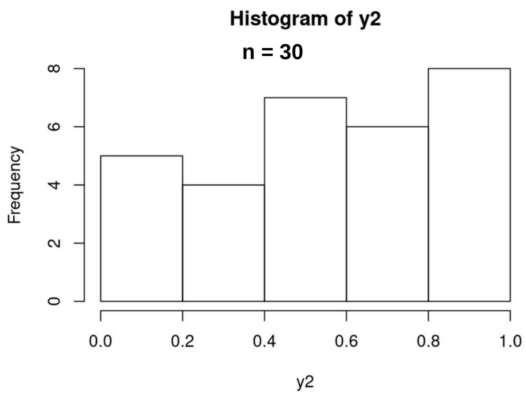
- glm(y ~ x, data = data)
- → and specify the distribution family
-
- glm(y ~ x, data = data, family = 'poisson')
- → look up available families with "?family" in R

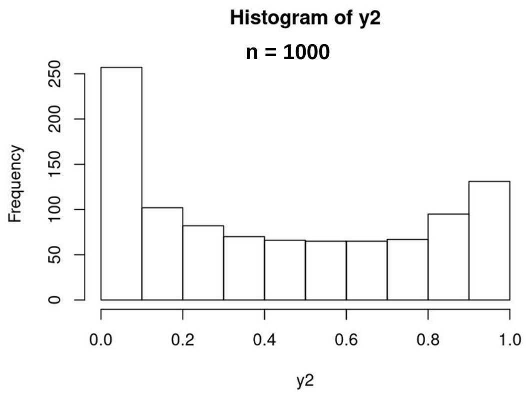
Case #2: data in range (0 → 1)



Case #2: data in range (0 → 1)

- Proportion data (%) (binomial distribution)
- Data tend to accumulate / squeeze a bit around 0 and 1



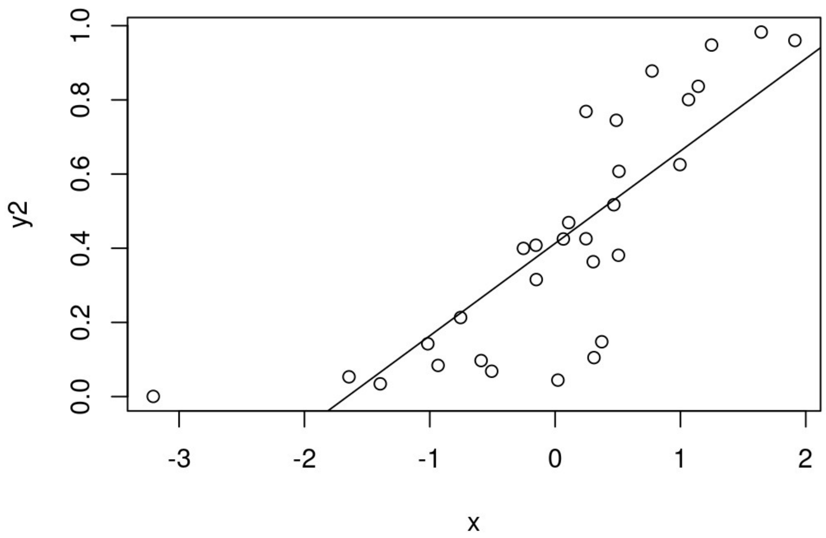


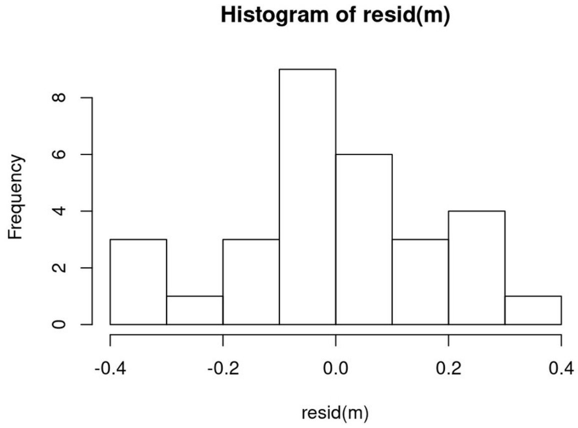
Case #2: data in range (0 → 1)

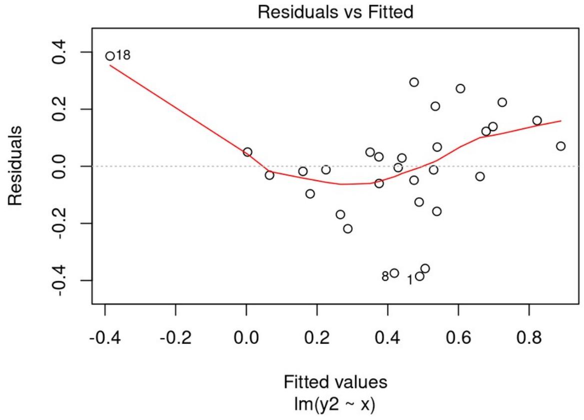
- → 0 and 1 are natural bounds
-
- Special case: (0, 1) data (absence / presence / occurrence data) (Bernoulli distribution)

Case #2: data in range (0 → 1)

- Again, a simple lm fit looks poor:







Case #2: data in range (0 → 1)

- How to deal with it?
- → logit transformation:
- y' = log(y / (1 – y))
- → 0.1 "becomes" -2.2, 0.9 "becomes" 2.2
- → lower and upper bounds are removed
- Does not work for y = exactly 0 or exactly 1
- (look up the formula for back-transformation)

Case #2: data in range (0 → 1)

- How to deal with it?
- Or use the family “binomial”:
- glm(y ~ x, data = data, family = ‘binomial’)

Other transformations

- Gaussian (normal), Poisson (log-normal), binomial (proportion data) and Bernoulli (0-1 data) have specific-use application
- Sometimes that information is not available and a normal distribution still does not fit well
- → What to do?

Other transformations

- i) Take square-root, or n-th root
- → how to take n-th root? n-th root = $y^{(1 / n)}$
-
- ii) Choose a different base for logarithmization
- → in R: “log(x, base = y)”
-
- iii) Transform both the y- and the x values

Other transformations - roots

- There is a caveat here – after taking the root, all values will be either all positive or all negative
- Root transformation therefore only works in some cases (e.g. when dealing with length-weight relationships)
- Log-transformation is not affected by this limitation

Try a different family

- Other families: e.g. Gamma, inverse-Gaussian, quasi-binomial, quasi-Poisson
-
- You can find the relevant transformation by assessing the goodness-of-fit of your model

Incorporate polynomials

- Quadratic term: x^2
- Cubic term: x^3
- x^4, x^5, …, x^n
- In glm() function: “poly(x, 2)”

Goodness-of-fit of your model

- Diagnostics plots
- Residual distribution (residuals of the transformed y-values)
- R^2 (of the transformed y-values)
- Careful with the AIC function: you can’t directly compare models based on y-values of different transformation
- → will be repeated / explored in more detail in the next lesson

Demonstration in R

Questions?

Exercise in R

- 1) You get bi-variate data sets (x and y) and need to find out the correct transformation(s) / the best model (each on their own)
- 2) We discuss your results

Exercise in R

- 3) In groups (2-3), you create your own bi-variate data sets
- y = transformation(*a* + *b* * *x* + random_error)
- *a* + *b* * *x* → you can implement a more complex formula
- Don’t add too much error
- Then you exchange them among groups, and, as in (1), try to find out the correct transformation(s) / best model

A little extra: non-linear mechanistic models

- LMs, GLMs are “context-less” models / purely data-driven models
- Useful when little to no prior information is available or very complex relationships are expressed in simpler statistical terms (e.g. ecosystem indicators)
- Mechanistic models: e.g. growth curves (phytoplankton experiments with Elisa), stock-recruitment curves

A little extra: non-linear mechanistic models

- Often contain quadratic or exponential terms
- e.g.:

  R = a * SSB * exp(-b * SSB)

- Providing starting values for parameters (a and b) is very important because parameter-search algorithm is affected by the exponential term

A little extra: non-linear mechanistic models

- Alternatively you can try to linearize the equation by rearrangement:

  $\log(R) = \log(a) + \log(SSB) + (-b * SSB)$

- "intercept" "slope"

- $\log(R) - \log(SSB) = \log(a) - b * SSB$

-

- For comparison glm equation: $y = a + b * x$

A little extra: non-linear mechanistic models

- For the parameter-estimation algorithm, it is then easier to find the optimum solution and you (often) don't need to provide start values
- In R: y ← log(R) – log(SSB)

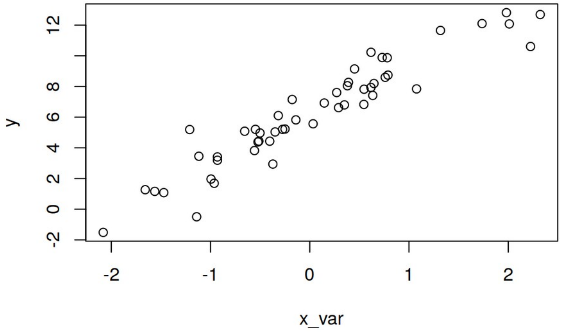
  nls(y ~ log_a – exp(log_beta) * SSB)

  -
  - The "exp()" is required because beta must not be negative

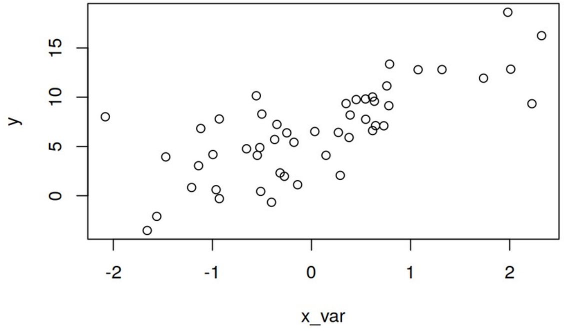
Extra: distributions of the residuals

- Regular assumption is that the residuals of a fit of a function are normally distributed
- But they can also assume different distributions
- If the details of the residual distribution are important, you will likely try to estimate the parameters of this distribution (e.g. sigma / the standard deviation) along with the model parameters

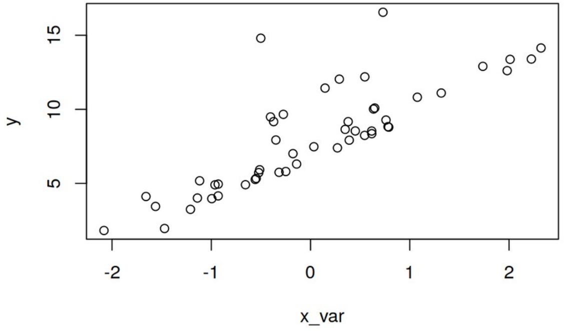
Residuals follow normal distribution with standard deviation = 1



Residuals follow normal distribution with standard deviation = 3

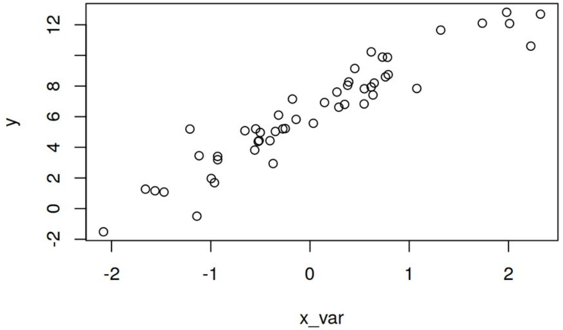


Residuals follow log-normal distribution

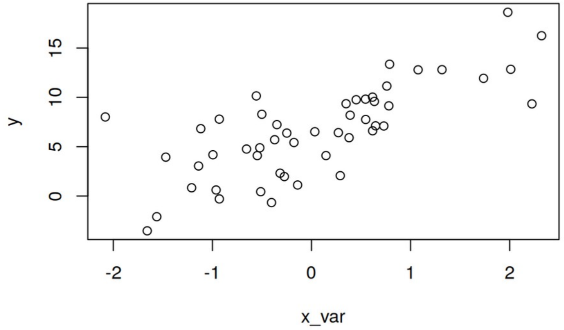


Extra: distributions of the residuals

Residuals follow normal distribution with standard deviation = 1



Residuals follow normal distribution with standard deviation = 3



Residuals follow log-normal distribution